

神威众核编程指南

2020 年 5 月 14 日

目 录

第一章 神威众核结构概述.....	1
1.1 芯片结构	1
1.2 从核阵列结构.....	1
1.3 从核结构	3
第二章 编程模型.....	1
2.1 众核程序结构.....	1
2.2 从核函数	1
2.3 从核组织方式.....	2
2.4 存储层次	2
2.4.1 主存层次	2
2.4.2 LDM 层次.....	3
第三章 编程接口.....	5
3.1 编译与运行.....	5
3.2 常用资源查询命令与作业提交选项.....	6
3.2.1 资源查询命令	6
3.2.2 常用作业提交选项	6
3.3 第一个众核程序.....	7
3.3.1 主函数	8
3.3.2 主核函数	9
3.3.3 从核函数	10
3.3.4 编译运行	12
3.4 主核与从核间数据传输.....	12
3.4.1 函数接口	13
3.4.2 程序示例	15
3.5 数据分块	17
第四章 从核性能优化	21
4.1 从核数据 CACHE	21
4.1.1 使用方法	21
4.1.2 程序示例	21
4.2 从核与从核间数据传输.....	22
4.2.1 函数接口	22
4.2.2 程序示例	24
4.3 LDM 共享.....	29
4.3.1 LDM 共享模式设置方法.....	31
4.3.2 使用方法	32
4.3.3 程序示例	32
4.4 手工 SIMD 优化.....	34
4.4.1 算法描述与接口	34
4.4.2 程序示例	35

4.5 自动 SIMD 优化.....	40
4.6 离散访存库.....	41
4.6.1 函数接口	41
4.6.2 程序示例	42

第一章 神威众核结构概述

神威众核芯片采用主从异构结构，由通用计算主核和精简的计算从核组成，具有超高的定点和浮点峰值性能和高的实际应用性能。

1.1 芯片结构

神威众核芯片集成 6 个核组，每个核组包括 1 个主核和 1 个从核阵列，1 个从核阵列包含 64 个从核；片上集成 2 个系统接口，上述主要部件通过环形网络连接；神威众核芯片的总体结构如下图 1-1 所示：

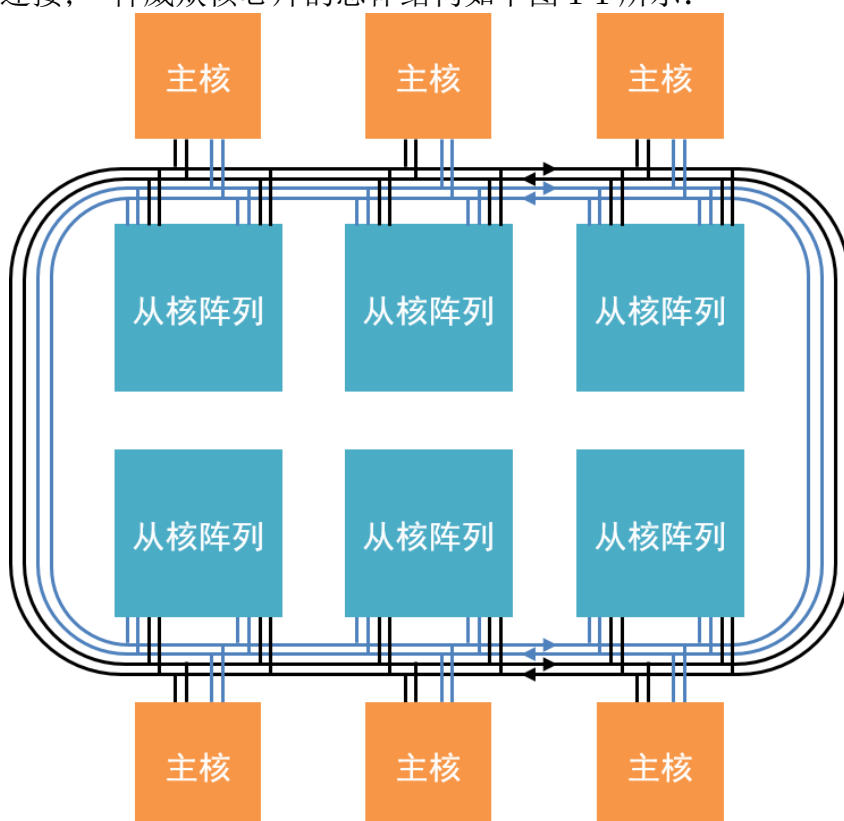
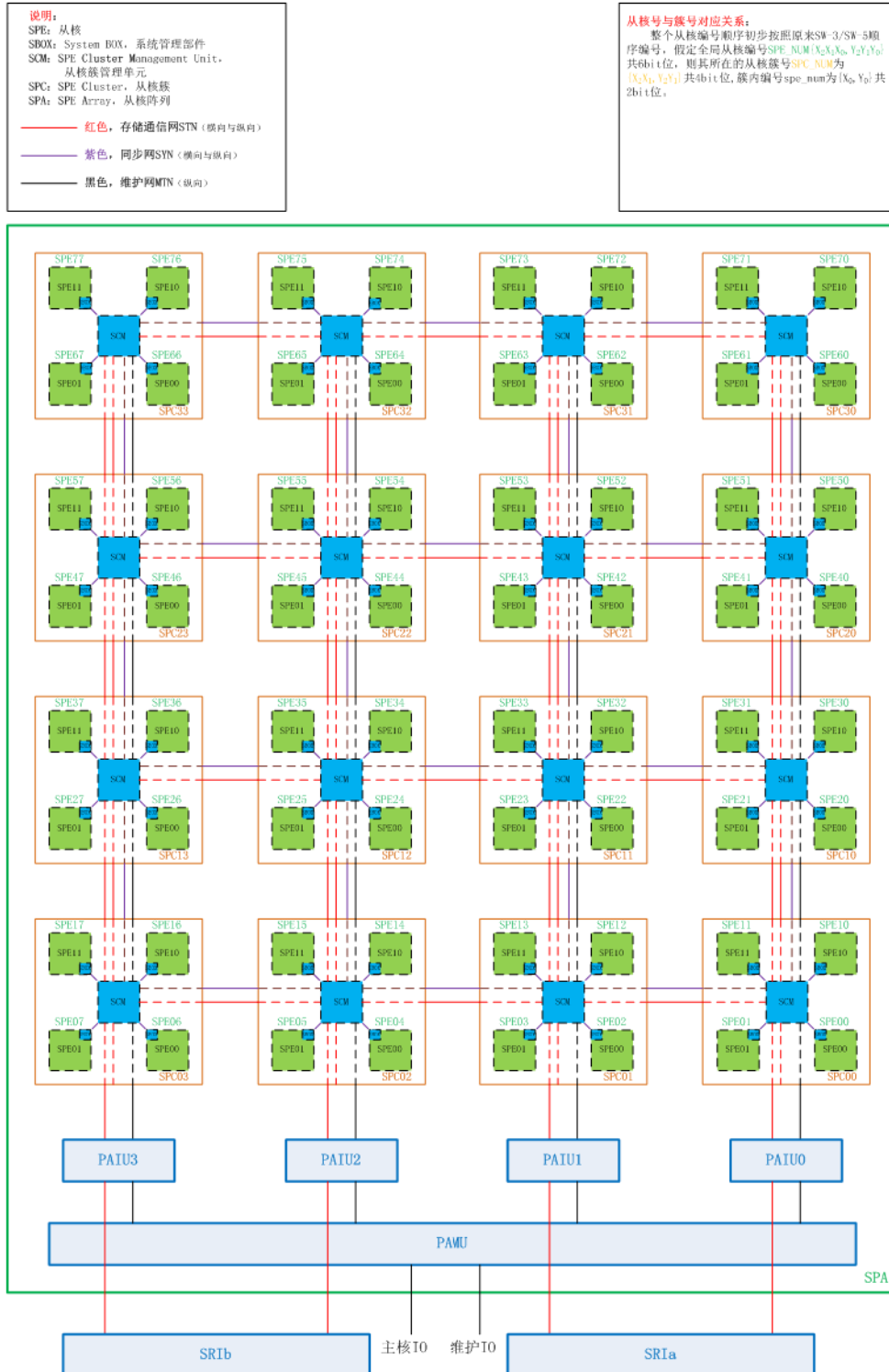


图 1-1 神威众核芯片基本图

1.2 从核阵列结构

神威众核芯片的从核以 8×8 阵列方式排布的从核阵列为基本单位进行管理，从核之间以及从核与外部的交互通过阵列内网络进行互连。从核之间的网络采用 CMesh 网络，四个临近从核共享一个从核簇管理部件，并集成一个路由器。神威众核芯片从核阵列结构如图 1-2 所示：



1.3 从核结构

神威众核处理器的从核使用一个 64×64 位的统一结构寄存器（SIMD 扩展结构使用高 32 个条目、512 位宽的向量寄存器），由整数执行部件和浮点执行部件共享，具体认作定点整数格式还是浮点数格式由程序员自行决定，所有运算均在寄存器之间进行。

支持字节（8 位）、半字（16 位）、字（32 位）和长字（64 位）的整数；支持 IEEE754 标准的半精度（16 位）、单精度（32 位）和双精度（64 位）浮点数据类型。

支持短向量数据，包括长度为 16 的字（ 16×32 位）向量整数、长度为 32 的半精度（ 32×16 位）向量浮点数据和长度为 8 的单精度（ 8×32 位）或双精度（ 8×64 位）向量浮点数据。

将 32 个 64 位的统一类型寄存器（\$32-\$63）扩展成为 32 个 512 位的向量寄存器，每个寄存器的低 64 位仍然是基本结构的统一类型寄存器，存放 8 位、16 位、32 位和 64 位的整数，以及半精度、单精度或双精度浮点数据，而整个 512 位的寄存器则可以存放 512 位的整数向量或浮点向量。从核寄存器结构如图 1-3 所示：

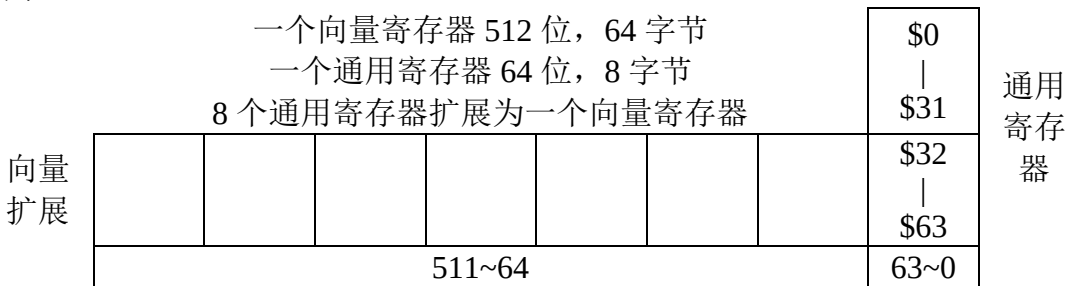


图 1-3 从核寄存器结构示意图

关于 SIMD 扩展和半精度支持详见《基础编译器 SIMD 用户编程手册》。

神威众核处理器的每个从核具有一块高速的本地局部数据存储空间 LDM，LDM 总容量为 256KB，该空间可配置成完全由软件管理，也可部分容量配置成硬件自动管理的数据 Cache，两种管理方式的容量配比分档可调。

配置成软件管理的局部存储时，从核可以使用访问 LDM 空间的 ld/st 的方式加以访问，也可以使用发起异步 DMA 或 RMA 的方式实现 LDM 与外界的数据批量交换；

配置 Cache 时，LDM 支持 0KB、32KB 和 128KB 三档数据 Cache 容量配置，通过提交作业时 bsub 选项-cache_size 0/32/128 进行配置。cache_size 默认为 0。

第二章 编程模型

2.1 众核程序结构

一个神威众核程序通常由三部分组成：

1、主函数

进行众核加速前的主程序，运行在纯主核上，调用神威主核函数进行众核加速；

2、神威主核函数

声明从核函数，设置从核函数参数，将从核函数 spawn 到从核阵列上并开始从核阵列计算。从核阵列计算开始后，主核函数等待所有从核完成计算并退出，在主核函数启动从核阵列后等待从核计算完成期间，主核函数可以执行与从核阵列异步执行运行的在主核运行的代码。

3、神威从核函数

由主核函数 spawn 到从核上由从核异步执行的函数。

2.2 从核函数

从核执行单独编写的从核函数，主核将同一个从核函数 spawn 到从核阵列上的每个从核，由从核进行计算，从核支持 C 和 Fortran，从核支持简单的 C++。一个从核函数在主核的声明如下：

```
...
extern SLAVE_FUN(MatrixMulSW)(void*);
...
```

主核函数 spawn 从核函数时可以将一个主存地址以参数的方式传递给从核函数，当需要向从核函数传递多个参数时，可以将多个参数声明为一个结构体，在主核函数中创建结构体变量并对结构体赋值，spawn 时将结构体变量的地址以参数的方式传给从核函数，从而实现向从核函数传递多个参数：

```
...
typedef struct{
    int hA;
    int wA;
    float *eA;
} Para;
...
para.hA = M;
para.wA = N;
para.eA = &A[0];
athread_spawn(func_slave, &para);
...
```

以上程序通过数据结构 Para 将变量 M，N 和 &A[0] 传入从核函数。

2.3 从核组织方式

每个从核簇由四个从核组成。从核为精简的 64 位 RISC 核心，支持 512 位整数和浮点 SIMD 操作。四个从核共享一个从核簇管理部件，主要负责阵列内网络的数据交互和流量控制，同时集成了用于批量访存的 DMA 引擎和从核批量数据交互的 RMA 引擎。从核簇管理部件还负责从核及簇管理部件自身的异常、错误、故障的报出，支持主核和维护对阵列内 IOR 的访问。其组织结构如图 1-2 所示。

可以通过直接使用以下变量获得从核在从核阵列和从核簇内的编号：

CRTS_tid 从核号（硬件编号，0-63）
CRTS_rid 从核所在行号（硬件编号，0-7）
CRTS_cid 从核所在列号（硬件编号，0-7）
CRTS_cgn 从核所在核组号（硬件编号，0-5）
CRTS_spc_tid 从核簇方式的逻辑从核号（0-63），可用于 LDM 共享
CRTS_spcn 从核簇号（0-15）

从核编号采用行列结构，阵列内从核编号具体如图 2-1 所示：

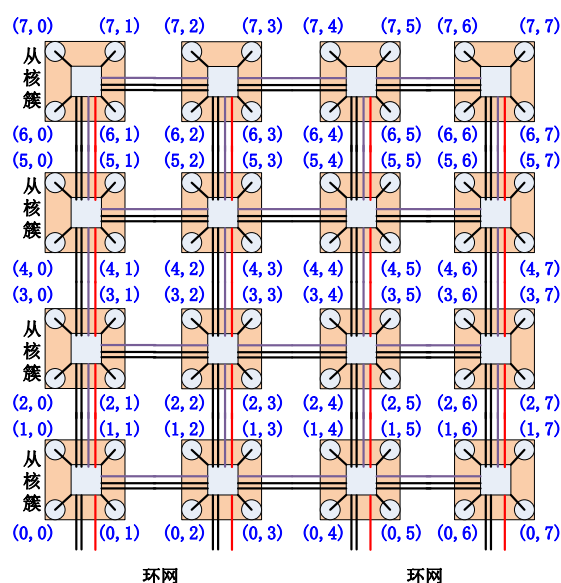


图 2-1 阵列内从核编号，括号内为（行号，列号）

2.4 存储层次

2.4.1 主存层次

神威众核芯片一个核组内的存储分为位于主核的主存和每个从核私有的位于每个从核的 LDM。编程时，从核函数栈空间默认位于主存中，在运行时加-b 选项可以将栈空间设置到 LDM 内。

2.4.2 LDM 层次

从核 LDM 空间可被划分为私有段、连续共享段和从核数据 Cache 空间三部分，三部分空间分档可调，LDM 使用时各空间的划分如图 2-3 所示。

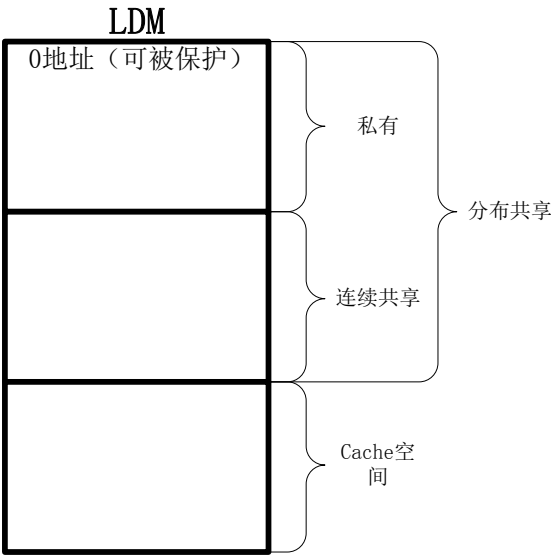


图 2-2 LDM 存储层次

支持的连续共享段长配置如下：

- 1、LDM 连续共享段长必须为 2 的幂，即每个从核内连续共享段的大小只能是 0、4、8、16、32、64、128KB；
- 2、除不配置 LDM 连续共享段，单从核阵列总的 LDM 连续共享段长不能小于单从核 LDM 私有段容量；
- 3、从核 LDM 私有段容量不能为 0。

具体配置如表 2-1 所示，表中“Dcache（从核数据 Cache）容量”、“LDM 连续共享段容量”由软件配置，“LDM 私有段容量”为总的局存容量（256KB）减去“Dcache（从核数据 Cache）容量”和“LDM 连续共享段容量”后剩余的大小；“LDM 分布共享容量”为总的局存容量（256KB）减去“Dcache 容量”后剩余的大小。

表 2-1 从核 LDM 空间合法的容量分配（单位：KB）

Dcache（从核数据 Cache）容量	LDM 连续共享段容量	LDM 私有段容量	LDM 分布共享段容量
0	0	256	256
	4	252	
	8	248	
	16	240	
	32	224	
	64	192	
	128	128	
32	0	224	224
	4	220	

	8	216	
	16	208	
	32	192	
	64	160	
	128	96	
128	0	128	128
	4	124	
	8	120	
	16	112	
	32	96	
	64	64	

有关 LDM 共享详细内容见 4.3 节。DMA、RMA 操作不能使用连续共享段的访问方式。

第三章 编程接口

3.1 编译与运行

神威众核程序由 3 部分组成：

- 1、运行于主核的主程序，通常为未使用众核加速的纯主核程序。
- 2、主核程序，包括从核函数声明、从核函数参数设置、从核函数的 spawn 与 join 和位于从核函数 spawn 之后、从核函数 join 之前的与从核函数异步执行的由主核运行的程序。
- 3、从核程序，包括 LDM 变量声明和从核函数实现。

神威众核程序的编译流程为：

- 1、编译主程序 main.c

```
sw9gcc -mftz -mieee main.c -o main.o
```

- 2、编译主核程序 master.c

```
sw9gcc -mftz -mieee master.c -o master.o
```

- 3、编译从核程序 slave.c

```
sw9gcc -mslave -mieee -msimd slave.c -o slave.o
```

- 4、链接生成可执行程序 exe

```
sw9gcc -mhybrid -static main.o master.o slave.o -o exe
```

详细编译选项说明参见《基础编译系统用户手册》。

众核程序需要使用 bsub 命令提交到计算节点运行，提交命令为：

```
bsub -b -I -o out.log -q 队列名 -shared -n 1 -cgsp 64 -share_size 14000 ./exe
```

编写和运行神威众核程序主要分为 4 个步骤：

- 1、使用 athread 线程库单独编写主核程序和从核程序；
- 2、用不同编译选项分别编译生成主核程序和从核程序的.o 文件；
- 3、使用混合链接选项链接生可执行程序；
- 4、将生成的可执行程序提交到计算结点进行计算

主核程序运行在主核上，可以包含多个从核函数，一个主核程序如下所示：

```
1  #include <crts.h>
2
3  extern SLAVE_FUN(func_slave)(void*);
4
5  void master(void *para){
6      athread_spawn(func_slave,para);
7      athread_join();
8  }
```

其中：

第 1 行：引用公共运行时（CRTS）库头文件；

第 3 行：声明外部从核函数，从核函数通常在从核程序文件中声明和定义；
 第 6 行：将从核函数 func_slave 加载到从核阵列上进行计算，函数 func_slave 的参数由 para 提供。

第 7 行：显式阻塞等待从核阵列完成计算；

在第 6 行（从核启动）和第 7 行（从核执行结束）之间可以添加主核代码，其代码在主核上与从核代码异步执行。

3.2 常用资源查询命令与作业提交选项

3.2.1 资源查询命令

资源查询命令为：

qload -w

输入命令后显示如下信息：

```

=====+
QUEUE_NAME          CONFIG ONLINE OFFLINE IDLE  BUSY  BOOT  SLEEP DOWN  SOFTFT HARDFT IDLEMPE IDLESPE  BUSYMPE BUSYSPE  USER/GROUP
=====+
q_debug_yyz         15      15      0      7      8      0      0      0      0      0      42      2496      33      2112      gfsyyz
=====+
  
```

其中，

QUEUE_NAME：队列名；

CONFIG：队列中配置的 CPU 总数；

IDLE：队列中空闲的 CPU 个数；

IDLEMPE：队列中空闲的主核个数，一个主核运行一个进程，此数即为可以提交的进程数。

注意：

一个 CPU 中通常包含 6 个核组（1 个核组为 1 个主核+64 个从核），因此 IDLE 和 IDLEMPE 通常为 6 倍关系。

3.2.2 常用作业提交选项

常用作业提交命令为：

bsub -b -I -o out.log -q 队列名 -shared -n 1 -cgsp 64 -share_size 15000 ./exe

其中，

-b：将从核栈空间放在 LDM 内。从核栈空间（在从核函数中声明的变量、从核文件中的函数等）默认位于该从核在主存内的从核私有段（详见 2.4 节），若不加-b 选项，则从核对从核栈变量的访问为从核访主存，加-b 选项后从核栈空间位于 LDM 内，则从核对从核栈变量的访问为本地 LDM 访问。

-I：交互式作业提交，不加-I 时为后台提交。当队列中空闲计算资源不足时可以通过后台提交作业方式将作业 PEND 到队列尾端，一旦队列中有足够的空闲资源时 PEND 的作业会自动提交运行。

-o: 将作业输出保存到指定的文件中。

-q: 指定作业提交的队列。

-shared: 以非独占方式提交作业。一个 CPU 内有 6 个主核，一个主核运行一个进程，因此，一个 CPU 应当可以运行 6 个串行作业，但若用独占方式提交作业，即在提交作业时不加-shared 选项时，即使作业仅使用 1 个主核（1 个进程），其它作业也无法提交到它运行的 CPU 上，它运行的 CPU 上其它 5 个主核处于空闲状态且无法运行任何作业直到 CPU 完全空闲。添加-shared 选项以非独占方式提交作业，当一个作业占用一个 CPU 内的 1 个主核时，依然可以该 CPU 的其它主核提交作业。

-n: 指定作业进程数。

-np: 每个 CPU 内运行的进程数。

-cgsp: 每个核组启动的从核数，只有添加此选项后主存内作为从核私有段的部分内存才可使用，因此即使作业不使用从核计算依然需要添加此选项。

-share_size: 指定作业使用的主存大小，以 MB 为单位。一个核组内存大小为 16GB，share_size 大小通常不超过 15000。作业使用的内存空间大小由此参数指定，无法动态查看作业运行时 CPU 内存使用情况。

-host_stack: 指定作业使用的主核栈空间，主核栈空间不应超过 share_size 指定的大小。

3.3 第一个众核程序

以矩阵矩阵 $C=A*B$ 乘为例，将矩阵 C 二维分成 64 个分块，每个从核计算 1 个分块，每个从核计算与 C 的分块相关的 A 的 1 块行和 B 的 1 块列，如图 3-1 所示。

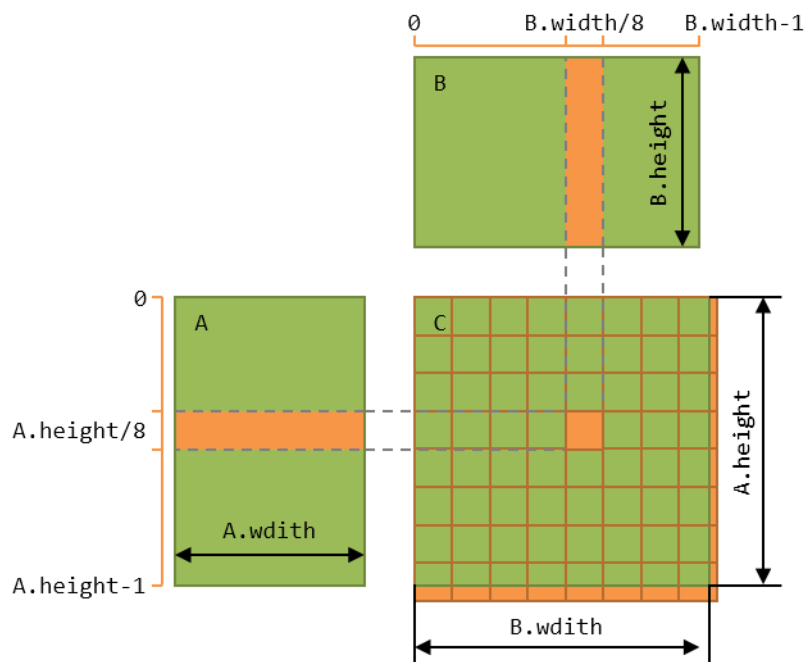


图 3-1 矩阵矩阵乘分块与计算

3.3.1 主函数

主函数位于 main.c 文件中，在主核运行，包括对 A、B、C 数组的空间申请和初始化，如下所示：

```
1  #include <crts.h>
2
3  #define BLOCK_SIZE 16
4  #define WA 4*BLOCK_SIZE
5  #define HA 4*BLOCK_SIZE
6  #define WB 5*BLOCK_SIZE
7
8  typedef struct {
9      int width;
10     int height;
11     int stride;
12     double* elements;
13 } Matrix;
14
15 extern void MatMul(Matrix *A, Matrix *B, Matrix *C);
16
17 int main(int argc, char* argv[]){
18     Matrix A, B, C;
19     int size_A, size_B, size_C;
20     int i, j;
21
22     CRTS_init();
23
24     A.width = WA;
25     A.height = HA;
26     B.width = WB;
27     B.height = WA;
28
29     C.width = B.width;
30     C.height = A.height;
31
32     /* Allocate A and B */
33     size_A = A.width * A.height;
34     A.elements = (double*)malloc(size_A*sizeof(double));
35     size_B = B.width * B.height;
36     B.elements = (double*)malloc(size_B*sizeof(double));
37
38     /* Allocate C */
39     size_C = C.width * C.height;
```

```

40     C.elements = (double*)malloc(size_C*sizeof(double));
41
42     /* set value of A and B */
43     for(i=0; i<size_A; i++)
44         A.elements[i] = 1.0f;
45     for(i=0; i<size_B; i++)
46         B.elements[i] = 0.01f;
47
48     /* coculate C */
49     MatMul(&A, &B, &C);
50
51     /* output reslut */
52     for(i=0; i<C.height; i++){
53         for(j=0; j<C.width; j++)
54             printf("%f ", C.elements[i]);
55         printf("\n");
56     }
57
58     free(A.elements);
59     free(B.elements);
60     free(C.elements);
61     athread_halt();
62     return 0;
63 }

```

其中：

第 1 行：引用神威众核公共运行时系统头文件，所有神威众核程序均需要引用此文件；

第 22 行：完成并行语言公共运行时的初始化，在使用主核同步和任何从核功能之前必须调用该函数，一个神威众核程序中调用且仅调用 1 次 CRTS_init() 函数；

第 61 行：释放由 CRTS_athread_spawn/athread_spawn 创建的线程组资源，停滞从核组流水线，关闭从核组，该从核组在本进程无法再次使用。

3.3.2 主核函数

主核函数位于 master.c 文件中，在主核运行，包括从核函数声明，从核函数参数设置，从核函数的启动和等待完成等，如下所示：

```

1     #include <crts.h>
2
3     typedef struct {
4         int width;
5         int height;
6         int stride;

```

```

7     double* elements;
8 } Matrix;
9
10 typedef struct{
11     int widthA;
12     int heightA;
13     int widthB;
14     int heightB;
15     double* eA;
16     double* eB;
17     double* eC;
18 } Para;
19
20 extern SLAVE_FUN(MatrixMulSW)(void*);
21
22 void MatMul(Matrix *A, Matrix *B, Matrix *C){
23     Para para;
24     para.widthA = A->width;
25     para.heightA = A->height;
26     para.widthB = B->width;
27     para.heightB = B->height;
28     para.eA = A->elements;
29     para.eB = B->elements;
30     para.eC = C->elements;
31
32     pthread_spawn(MatrixMulSW, &para);
33     pthread_join();
34 }

```

其中：

第 10~18 行：声明从核函数参数数据结构，用于向从核函数传递多个参数；

第 20 行：声明从核函数；

第 23~30 行：声明从核函数参数数据结构变量并对其赋值；

第 32 行：启动从核函数，将从核函数参数数据结构变量地址传入从核函数；

第 33 行：等待从核阵列运行完成，在 32 行和 33 行之间可以添加在主核运行与从核函数异步执行的程序。

3.3.3 从核函数

从核函数位于 slave.c 中，由每个从核异步执行，如下所示：

```

1  #include <crts.h>
2
3  #define BLOCK_SIZE 16
4  #define WA 4*BLOCK_SIZE

```

```

5  #define HA 4*BLOCK_SIZE
6  #define WB 5*BLOCK_SIZE
7
8  typedef struct {
9      int width;
10     int height;
11     int stride;
12     double* elements;
13 } Matrix;
14
15 typedef struct{
16     int widthA;
17     int heightA;
18     int widthB;
19     int heightB;
20     double* eA;
21     double* eB;
22     double* eC;
23 } Para;
24
25 #define WC WB/8
26 #define HC HA/8
27
28 __thread_local double C_s[HC][WC] __attribute__((aligned(64)));
29
30 __thread_local crts_rply_t dma_rply = 0;
31 __thread_local unsigned int D_COUNT = 0;
32
33 void MatrixMulSW(void* para){
34     Para para_s;
35     int rstartC, cstartC;
36     int row, col; /* global index of C_s[i][j] */
37     int wA, wB;
38     int i, j, e;
39
40     rstartC = CRTS_rid * HC;
41     cstartC = CRTS_cid * WC;
42
43     /* 1. get parameters from host */
44     CRTS_dma_iget(&para_s, para, sizeof(Para), &dma_rply);
45     D_COUNT++;
46     CRTS_dma_wait_value(&dma_rply, D_COUNT);
47
48     wA = para_s.widthA;

```

```

49     wB = para_s.widthB;
50
51     for(i=0; i<HC; i++)
52         for(j=0; j<WC; j++)
53             C_s[i][j] = 0.0;
54
55     /* 2. coculate C */
56     for(i=0; i<HC; i++){
57         row = rstartC + i;
58         for(j=0; j<WC; j++){
59             col = cstartC + j;
60             for(e=0; e<wA; e++){
61                 C_s[i][j] += para_s.eA[row * wA + e]
62                             * para_s.eB[e * wB + col];
63             }
64         }
65     }
66
67     /* 3. put C back to host */
68     CRTS_dma_iput_stride(para_s.eC+rstartC*wB+cstartC, &C_s[0][0],
69         HC*WC*sizeof(double), WC*sizeof(double), (wB-WC)*sizeof(double),
70         &dma_rply);
71     D_COUNT++;
72     CRTS_dma_wait_value(&dma_rply, D_COUNT);
73 }

```

3.3.4 编译运行

编译链接命令如下：

```
sw9gcc -mftz -mieee main.c -o main.o
```

```
sw9gcc -mftz -mieee master.c -o master.o
```

```
sw9gcc -mslave -mieee -msimd slave.c -o slave.o
```

```
sw9gcc -mhybrid -static main.o master.o slave.o -o exe
```

运行命令如下：

```
bsub -b -I -o out.log -q 队列名 -shared -n 1 -cgsp 64 -share_size 15000 ./exe
```

3.4 主核与从核间数据传输

注意到 3.3 节矩阵矩阵乘程序中从核函数中的第 61~62 行：

```

.....
61         C_s[i][j] += para_s.eA[row * wA + e]
62                 * para_s.eB[e * wB + col];
.....

```

每个从核计算一个 C 小块时，其对 A 和 B 的访问是对主存地址的直接访存，在神威众核芯片中，从核对主存地址的访存可以通过 gld/gst 方式或从核数据 Cache 方式从硬件自动实现，从核数据 Cache 的使用详见 4.1 节。对 A 和 B 的访问还可以通过 DMA 的方式将 A 和 B 的数据加载到 LDM，将对 A 和 B 的访问由直接访问主存转换为对本地 LDM 的访问。

3.4.1 函数接口

int CRTS_dma_iget(void *dst, void *src, int len, crts_rply_t *rply);

函数说明：

单个从核发起从主存到本地局存（LDM）的 DMA 数据传输，非阻塞方式，通常与函数 CRTS_dma_wait_value() 配合使用。

参数说明：

dst: 本地局存（LDM）目标地址（4B 对界/对齐）；

src: 主存源地址（4B 对界/对齐）；

len: 数据传输量（4B 整数倍）；

rply: 回答字地址，必须局存地址。

返回值：

成功：0；

失败：非 0。

int CRTS_dma_iget_stride(void *dst, void *src, int len, int bsize, int stride, crts_rply_t *rply);

函数说明：

单个从核发起从主存到本地局存（LDM）的 DMA 数据传输，非阻塞方式，通常与函数 CRTS_dma_wait_value() 配合使用。

参数说明：

dst: 本地局存（LDM）目标地址（4B 对界/对齐）；

src: 主存源地址（4B 对界/对齐）；

len: 数据传输量（4B 整数倍）；

bsize: 跨步向量块大小（4B 整数倍）；

stride: 主存跨步（4B 整数倍），即两个相邻跨步向量块之间的空白间隙；

rply: 回答字地址，必须局存地址。

返回值：

成功：0；

失败：非 0。

int CRTS_dma_iput(void *dst, void *src, int len, crts_rply_t *rply);

函数说明：

单个从核发起从本地局存（LDM）到主存的 DMA 数据传输，非阻塞方式，通常与函数 CRTS_dma_wait_value()配合使用。

参数说明：

dst: 主存目标地址（4B 对界/对齐）；
src: 本地局存源地址（4B 对界/对齐）；
len: 数据传输量（4B 整数倍）；
rply: 回答字地址，必须局存地址；

返回值：

成功：0；
失败：非 0。

int CRTS_dma_iput_stride(void *dst, void *src, int len, int bsize, int stride, crts_rply_t *rply);

函数说明：

单个从核发起从本地局存到主存的 DMA 数据传输，非阻塞方式，通常与函数 CRTS_dma_wait_value()配合使用。

参数说明：

dst: 主存目标地址（4B 对界/对齐）；
src: 本地局存源地址（4B 对界/对齐）；
len: 数据传输量（4B 整数倍）；
bsize: 跨步向量块大小（4B 整数倍）；
stride: 主存跨步（4B 整数倍），即两个相邻跨步向量块之间的空白间隙；
rply: 回答字地址，必须局存地址。

返回值：

成功：0；
失败：非 0。

int CRTS_dma_wait_value(crts_rply_t *rply, int value);

函数说明：

该函数用于判断相应的 DMA 操作是否完成。当(*rply)的值大于等于 value 时，表明相应操作已经完成，即：

若调用该函数的从核为数据获取方，表明数据已被写入到本从核的 LDM；

若调用该函数的从核为数据供应方，表明数据已从本从核的 LDM 取走。

参数说明：

rply: DMA 传输回答字地址，必须局存地址（LDM）；
value: 需等待完成的非阻塞 DMA 次数，为自然数。

返回值：

成功：0；

失败：非 0。

完整的接口说明见《并行语言公共运行时系外部接口》。

3.4.2 程序示例

使用 DMA 方式加载 A 和 B 的从核函数如下：

```
1  #include <crt.h>
2
3  #define BLOCK_SIZE 16
4  #define WA 4*BLOCK_SIZE
5  #define HA 4*BLOCK_SIZE
6  #define WB 5*BLOCK_SIZE
7
8  typedef struct {
9      int width;
10     int height;
11     int stride;
12     double* elements;
13 } Matrix;
14
15 typedef struct{
16     int widthA;
17     int heightA;
18     int widthB;
19     int heightB;
20     double* eA;
21     double* eB;
22     double* eC;
23 } Para;
24
25 #define WC WB/8
26 #define HC HA/8
27
28 __thread_local double C_s[HC][WC] __attribute__((aligned(64)));
29 __thread_local double A_s[HC][WA] __attribute__((aligned(64)));
30 __thread_local double B_s[WA][WC] __attribute__((aligned(64)));
31
32 __thread_local crt_rply_t dma_rply = 0;
33 __thread_local unsigned int D_COUNT = 0;
34
35 void MatrixMulSW(void* para){
36     Para para_s;
37     int rstartC, cstartC;
```

```

38     int wA, wB;
39     int i, j, e;
40
41     rstartC = CRTS_rid * HC;
42     cstartC = CRTS_cid * WC;
43
44     /* 1. get parameters from host */
45     CRTS_dma_iget(&para_s, para, sizeof(Para), &dma_rply);
46     D_COUNT++;
47     CRTS_dma_wait_value(&dma_rply, D_COUNT);
48
49     wA = para_s.widthA;
50     wB = para_s.widthB;
51
52     for(i=0; i<HC; i++)
53         for(j=0; j<WC; j++)
54             C_s[i][j] = 0.0;
55
56     /* 2. get A_s and B_s from host */
57     CRTS_dma_iget(A_s, para_s.eA+rstartC*wA, HC*WA*sizeof(double),
58         &dma_rply);
59     CRTS_dma_iget_stride(B_s, para_s.eB+cstartC, WA*WC*sizeof(double),
60         WC*sizeof(double), (wB-WC)*sizeof(double), &dma_rply);
61     D_COUNT += 2;
62     CRTS_dma_wait_value(&dma_rply, D_COUNT);
63
64     /* 3. coculate C */
65     for(i=0; i<HC; i++){
66         for(j=0; j<WC; j++){
67             for(e=0; e<WA; e++){
68                 C_s[i][j] += A_s[i][e]
69                     * B_s[e][j];
70             }
71         }
72     }
73
74     /* 3. put C back to host */
75     CRTS_dma_iput_stride(para_s.eC+rstartC*wB+cstartC, &C_s[0][0],
76         HC*WC*sizeof(double), WC*sizeof(double), (wB-WC)*sizeof(double),
77         &dma_rply);
78     D_COUNT++;
79     CRTS_dma_wait_value(&dma_rply, D_COUNT);
80 }

```

程序中声明了两个 LDM 数组 A_s 和 B_s，用于存放该从核计算时需要的 A

行块和 B 列块，如图 3-1 所示，通过程序第 57~62 行用 DMA 将需要的 A 行块和 B 列块加载到 LDM，注意由于 C 语言中数组按行优先存储，因此对加载 B 列块时需要跨步 DMA。完成数据加载后，对每个 C 分块的计算（第 68~69 行）就由原来的访问主存地址改为访问 LDM。

3.5 数据分块

在实际应用中，矩阵 A 和 B 的数据规模通常很大，无法一次将一个 C 分块计算所需要的 A 行块和 B 列块完全加载进从核 LDM，此时，需要对 A 行块和 B 列块进行分块，分次向 LDM 加载一个大小为 $HMAX \times BAMAX$ 和大小为 $BAMAX \times WMAX$ 的分块，分批加载进从核 LDM 分批计算。每次分块计算得到一个大小为 $HMAX \times WMAX$ 的 C 分块，因此需要对 C 重新分块。一个对 A、B、C 分块的示意如图 3-2 所示。

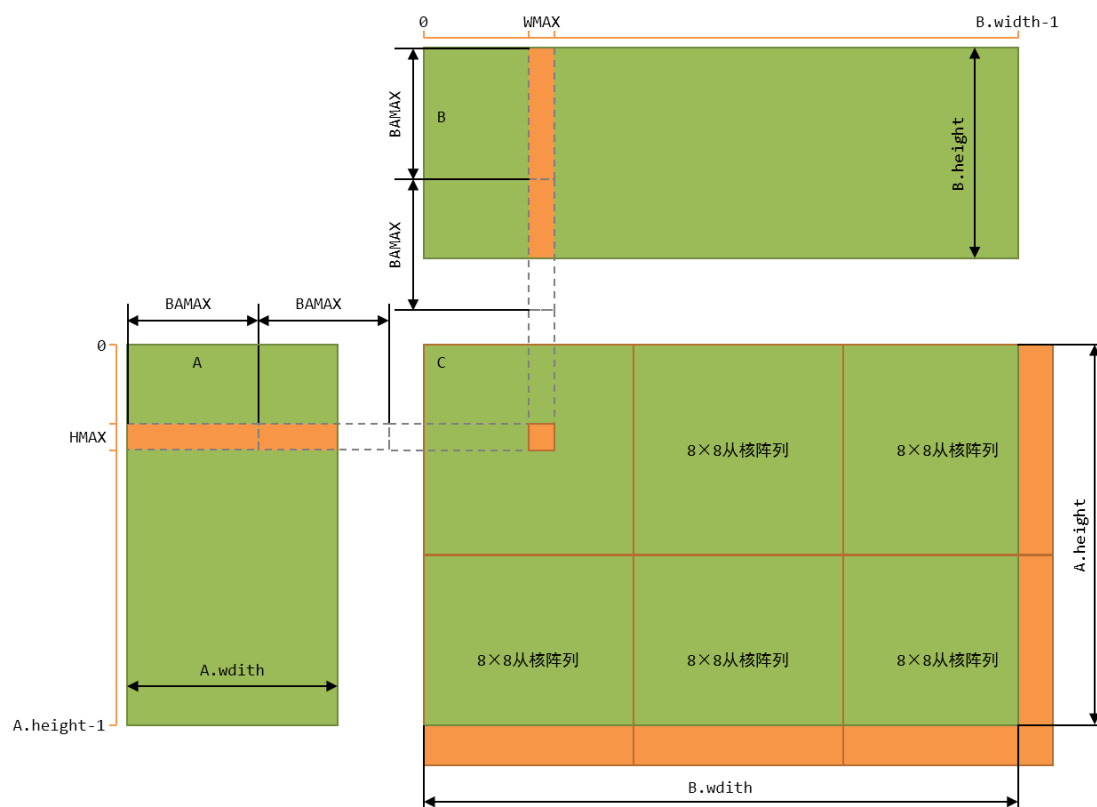


图 3-2 矩阵矩阵乘 DMA 分块示意图

每个从核计算一个大小为 $HMAX \times WMAX$ 的 C 的分块，对 C 的分块如图 3-3 所示：

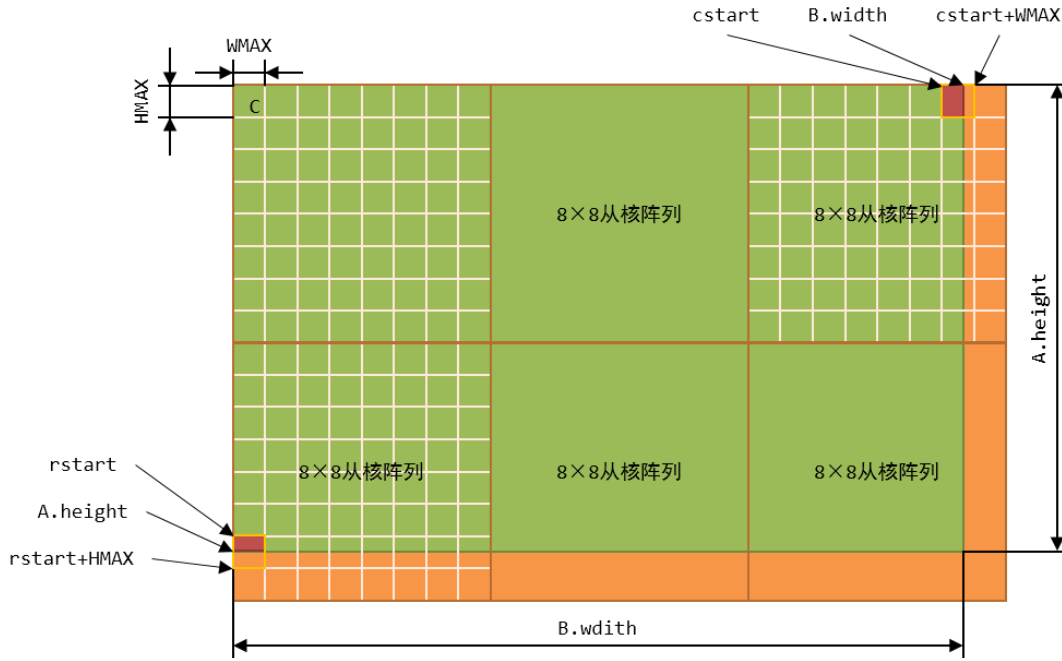


图 3-3 对 C 分块示意图

对应从核代码为:

```

1  .....
2  hblock = hA / 8;
3  if(hblock > HMAX) hblock = HMAX;
4  wblock = wB / 8;
5  if(wblock > WMAX) wblock = WMAX;
6
7  /* block C */
8  for(rstartC=CRTS_rid*hblock; rstartC<hA; rstartC+=(hblock*8)){
9      iend = hblock;
10     if(rstartC+iend > hA) iend = hA - rstartC;
11
12     for(cstartC=CRTS_cid*wblock; cstartC<wB; cstartC+=(wblock*8)){
13         jend = wblock;
14         if(cstartC+jend > wB) jend = wB - cstartC;
15
16         .....
17
18         /* put C back to host */
19         CRTS_dma_iput_stride(para_s.eC+rstartC*wB+cstartC, C_s,
20 iend*jend*sizeof(double), jend*sizeof(double), (wB-jend)*sizeof(double),
21 &dma_rply);
22         D_COUNT++;
23         CRTS_dma_wait_value(&dma_rply, D_COUNT);
24     }
25 }

```

在计算每个 C 分块里，需要大小为 $HMAX \times A.width$ 的 A 的行分块和大小

为 $B.height \times WMAX$ 的 B 的列分块，由于在矩阵矩阵乘中， $A.width$ 与 $B.height$ 相等，因此，可以对每个 C 分块需要的 A 的行分块和 B 的列分块分别按大小 $HMAX \times BAMAX$ 和 $BAMAX \times WMAX$ 分块，如图 3-2 所示，对应的从核代码为：

```

1  .....
2  hblock = hA / 8;
3  if(hblock > HMAX) hblock = HMAX;
4  wblock = wB / 8;
5  if(wblock > WMAX) wblock = WMAX;
6
7  /* block C */
8  for(rstartC=CRTS_rid*hblock; rstartC<hA; rstartC+=(hblock*8)){
9      iend = hblock;
10     if(rstartC+iend > hA) iend = hA - rstartC;
11
12     for(cstartC=CRTS_cid*wblock; cstartC<wB; cstartC+=(wblock*8)){
13         jend = wblock;
14         if(cstartC+jend > wB) jend = wB - cstartC;
15
16         /* init C */
17         .....
18
19         /* block A row block and B column block */
20         for(ibA=0; ibA<wA; ibA+=BAMAX){
21             bA = BAMAX;
22             if(ibA+bA > wA) bA = wA - ibA;
23
24             /* 2. get A_s and B_s from host */
25             CRTS_dma_iget_stride(A_s, para_s.eA+rstartC*wA+ibA,
26 iend*bA*sizeof(double), bA*sizeof(double), (wA-bA)*sizeof(double),
27 &dma_rply);
28             CRTS_dma_iget_stride(B_s, para_s.eB+cstartC+ibA*wB,
29 bA*jend*sizeof(double), jend*sizeof(double), (wB-jend)*sizeof(double),
30 &dma_rply);
31             D_COUNT += 2;
32             CRTS_dma_wait_value(&dma_rply, D_COUNT);
33
34             /* 3. coculate C */
35             for(i=0; i<iend; i++){
36                 for(j=0; j<jend; j++){
37                     for(e=0; e<bA; e++){
38                         C_s[i*jend+j] += A_s[i*bA+e]
39                                     * B_s[e*jend+j];
40                     }

```

```

41         }
42     }
43
44 }
45
46     /* 4. put C back to host */
47     CRTS_dma_iput_stride(para_s.eC+rstartC*wB+cstartC, C_s,
48 iend*jend*sizeof(double), jend*sizeof(double), (wB-jend)*sizeof(double),
49 &dma_rply);
50     D_COUNT++;
51     CRTS_dma_wait_value(&dma_rply, D_COUNT);
52 }
53 }

```

以上代码段中加粗部分即为实现对 A 的行块和 B 的列块进行分块后分批加载到 LDM 中。

第四章 从核性能优化

4.1 从核数据 Cache

如 2.4.2 节所述，可以将从核 LDM 的一部分空间配置为从核 Cache，配置 Cache 时，LDM 支持 0KB、32KB 和 128KB 三档数据 Cache。从核数据 Cache 支持硬件淘汰和装填，一致性的相关说明如下：

1、从核 Cache 不保证与主核 Cache 的一致性。主核访存过程中无法从从核 Cache 中获得最新数据；但从核淘汰或装填时，能够获取主核 Cache 中的最新数据；

2、从核 Cache 不保证与从核 Cache 的一致性。从核访存过程中，始终无法从其他从核的 Cache 中获得最新数据；

3、从核 Cache 不保证与软件管理的 LDM 空间的一致性。从核 DMA 过程中，无法获得从核数据 Cache 中的最新数据；

4、为避免由于从核 Cache 不保证一致性带来的假共享问题，从核 Cache 对共享主存空间(从核主存虚空间中的共享段)的写回支持带屏蔽的细粒度写机制，屏蔽粒度为 4B，一个 Cache 行中只有确实被修改过的数据才会写入主存(Dcache 写主存过程中，如果存在写数据带部分屏蔽，存控对相应的 128B 或 64B 采用读改写方式处理)；

5、对于“从核 Cache-从核 Cache”、“从核 Cache-DMA 操作访问”、“从核 Cache-同一片主存空间的不可 Cache 访问”或“从核 Cache-主核 Cache”之间存在真共享数据且最新副本在从核/主核 Cache 中，或“从核 Cache-从核 Cache”之间存在独占 Cache 行所有权限转移时，软件必须显式对数据所有者从核/主核的 Cache 进行刷新，通过 Memb 保证刷新完成，最后双方通过硬件同步或主存空间上的原子操作进行同步。

4.1.1 使用方法

通过提交命令 (bsub) 的 -cache_size 选项进行配置和使用，设置从核数据 Cache 后，从 LDM 的尾部开辟一块指定大小的空间作为从核数据 Cache，如下：

-cache_size 0/32/128，单位为 KB，其中：

-cache_size 0：从核数据 Cache 大小为 0KB，即不使用从核数据 Cache；

-cache_size 32：从核数据 Cache 大小为 32KB；

-cache_size 128：从核数据 Cache 大小为 128KB。

不使用 -cache_size 选项时，从核数据 Cache 大小默认设置为 0KB，即从核数据 Cache 默认关闭，使用 -cache_size 选项并且大小不为 0 时，从核内的直接访主存操作自动通过从核数据 Cache 实现。

4.1.2 程序示例

以 3.3 节矩阵矩阵乘程序为例，程序中从核函数中的第 61~62 行：

```
.....
61          C_s[i][j] += para_s.eA[row * wA + e]
62          * para_s.eB[e * wB + col];
.....
```

当在提交命令中设置 `cache_size` 大小不为 0 时，其对主存数组 `para_s.eA` 和 `para_s.eB` 的访问自动通过从核数据 Cache 实现，使用从核数据 Cache 的运行命令如下：

```
bsub -b -I -o out.log -q 队列名 -shared -n 1 -cgsp 64 -share_size 15000 -cache_size 32 ./exe
```

或

```
bsub -b -I -o out.log -q 队列名 -shared -n 1 -cgsp 64 -share_size 15000 -cache_size 128 ./exe
```

4.2 从核与从核间数据传输

一个从核可以通过 RMA 的方式发起对其它从核 LDM 的访问，从而实现从核与从核间的数据传输。

4.2.1 函数接口

int CRTS_rma_iget(void *l_addr, crts_rply_t *l_rply, int len, int r_tid, void *r_addr, crts_rply_t *r_rply);

函数说明：

以非阻塞方式发起 RMA 读操作，通常与函数 `CRTS_rma_wait_value()` 配合使用。

参数说明：

`l_addr`: 本地接收地址，必须为局存地址（4B 对界/对齐）；

`l_rply`: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成；

`len`: 数据传输量，单位为字节（4B 的整数倍）；

`r_tid`: 远程从核号；

`r_addr`: 远程源地址，必须为局存地址（4B 对界/对齐）；

`r_rply`: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值：

成功：0；

失败：非 0。

int CRTS_rma_iput(void *l_addr, crts_rply_t *l_rply, int len, int r_tid, void *r_addr, crts_rply_t *r_rply);

函数说明：

以非阻塞方式发起 RMA 写操作，通常与函数 `CRTS_rma_wait_value()` 配合使用。

参数说明：

`l_addr`: 本地接收地址，必须为局存地址（4B 对界/对齐）；

`l_rply`: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成；

len: 数据传输量，单位为字节（4B 的整数倍）；
r_tid: 远程从核号；
r_addr: 远程源地址，必须为局存地址（4B 对界/对齐）；
r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值：

成功：0；

失败：非 0。

int CRTS_rma_wait_value(crts_rply_t *rply, int value);

函数说明：

该函数用于判断相应的 RMA 操作是否完成。当(*rply)的值大于等于 value 时，表明相应操作已经完成，即：

若调用该函数的从核为数据获取方，表明数据已被写入到本从核的 LDM；

若调用该函数的从核为数据供应方，表明数据已从本从核的 LDM 取走。

参数说明：

rply: 回答字地址，必须局存地址；

value: 需等待完成的非阻塞 RMA 次数，为自然数。

返回值：

成功：0；

失败：非 0。

void CRTS_ssyc_array(void);

函数说明：

从核核组同步。

参数说明：

无。

返回值：

无。

void CRTS_ssyc_peer(int tid);

函数说明：

从核点对点同步，即本从核与本核组内编号为 tid 的从核同步。同步的 2 个从核调用该函数时，必须互相以对方的从核编号为参数。

参数说明：

int tid: 本核组内与本从核同步的从核编号。

返回值：

无。

void CRTS_ssyc_row(void);

函数说明：

核组内编号除 8 后相等的 8 个从核同步，如 0-7 号从核，16-23 号从核。

参数说明：

无。

返回值：

无。

void CRTS_ssyc_col(void);

函数说明：

核组内编号模 8 后相等的 8 个从核同步，如 0/8/16/24/32/40/48/56 号从核。

参数说明：

无。

返回值：

无。

完整的接口说明见《并行语言公共运行时系外部接口》。

4.2.2 程序示例

以一个 128 个 double 元素向量在从核阵列上的归约为例，其众核程序示例如下

主函数程序

主函数程序位于 main.c 中，内容如下：

```
1  #include <crts.h>
2
3  typedef struct{
4      double *a;
5      double *sum;
6  } Para;
7
8  extern void master(void *para);
9
10 int main(int argc, char** argv){
11     double *a;
12     double sum;
13     int n = 128;
14     int i;
```

```
15     Para para;
16
17     CRTS_init();
18
19     a = (double*)malloc(n*sizeof(double));
20     for(i=0; i<n; i++){
21         a[i] = 1.0*i + 0.001*i;
22     }
23     sum = 0.0;
24     para.a = a;
25     para.sum = &sum;
26
27     master(&para);
28
29     printf("slave  sum=%lf\n",sum);
30     sum = 0.0;
31     for(i=0; i<n; i++){
32         sum += a[i];
33     }
34     printf("master sum=%lf\n",sum);
35
36     free(a);
37     athread_halt();
38     return 0;
39 }
```

主核程序

主核程序位于 master.c 中，内容如下：

```
1  #include <crt.h>
2
3  extern SLAVE_FUN(func_slave)(void*);
4
5  void master(void *para){
6      athread_spawn(func_slave,para);
7      athread_join();
8  }
```

从核程序

从核程序位于 slave.c 中，内容如下：

```
1  #include <crts.h>
2
3  typedef struct{
4      double *a;
5      double *sum;
6  } Para;
7
8  __thread_local double a_s[2] __attribute__((aligned(64)));
9  __thread_local double buf_s __attribute__((aligned(64)));
10 __thread_local double sum_s __attribute__((aligned(64)));
11 __thread_local crts_rply_t dma_rply = 0;
12 __thread_local unsigned int D_COUNT = 0;
13 __thread_local crts_rply_t rma_rplyl = 0;
14 __thread_local crts_rply_t rma_rplyr = 0;
15 __thread_local unsigned int R_COUNTL = 0;
16 __thread_local unsigned int R_COUNTR = 0;
17
18 void func_slave(void* para){
19     int pe_id, row_id, col_id, pep;
20     Para para_s;
21     unsigned int flag, flag2;
22
23     pe_id = CRTS_tid;
24     row_id = CRTS_rid;
25     col_id = CRTS_cid;
26
27     /* 1. get parameters from host */
28     CRTS_dma_iget(&para_s, para, sizeof(Para), &dma_rply);
29     D_COUNT++;
30     CRTS_dma_wait_value(&dma_rply, D_COUNT);
31
32     /* 2. get a from host */
33     CRTS_dma_iget(a_s, para_s.a+2*pe_id, 2*sizeof(double), &dma_rply);
34     D_COUNT++;
35     CRTS_dma_wait_value(&dma_rply, D_COUNT);
36
37     /* 3. coculate the sum inside each slave */
38     sum_s = a_s[0] + a_s[1];
39
40     /* 4. even columns get odd columns and sum */
```

```

41     CRTS_ssync_array();
42     if(col_id%2 == 0){
43         CRTS_rma_iget(&buf_s, &rma_rplyl, sizeof(double), pe_id+1, &sum_s,
&rma_rplyr);
44         R_COUNTL++;
45         CRTS_rma_wait_value(&rma_rplyl, R_COUNTL);
46
47         sum_s += buf_s;
48     }else{
49         R_COUNTR++;
50         CRTS_rma_wait_value(&rma_rplyr, R_COUNTR);
51     }
52
53     /* 5. 0, 4 columns get from 2, 6 columns and sum */
54     if(col_id==0 || col_id==4){
55         CRTS_ssync_peer(pe_id+2);
56         CRTS_rma_iget(&buf_s, &rma_rplyl, sizeof(double), pe_id+2, &sum_s,
&rma_rplyr);
57         R_COUNTL++;
58         CRTS_rma_wait_value(&rma_rplyl, R_COUNTL);
59
60         sum_s += buf_s;
61     }else if(col_id==2 || col_id==6){
62         CRTS_ssync_peer(pe_id-2);
63         R_COUNTR++;
64         CRTS_rma_wait_value(&rma_rplyr, R_COUNTR);
65     }
66
67     /* 6. 0 columns get from 4 columns and sum */
68     if(col_id == 0){
69         CRTS_ssync_peer(pe_id+4);
70         CRTS_rma_iget(&buf_s, &rma_rplyl, sizeof(double), pe_id+4, &sum_s,
&rma_rplyr);
71         R_COUNTL++;
72         CRTS_rma_wait_value(&rma_rplyl, R_COUNTL);
73
74         sum_s += buf_s;
75     }else if(col_id == 4){
76         CRTS_ssync_peer(pe_id-4);
77         R_COUNTR++;
78         CRTS_rma_wait_value(&rma_rplyr, R_COUNTR);
79     }
80
81     /* 7. sum the sums in column 0 */

```

```

82     CRTS_ssync_col();
83     if(col_id == 0){
84         for(flag=1; flag<8; flag=(flag<<1)+1){
85             flag2 = flag >> 1;
86             pep = (flag2+1) * 8;
87             if(!flag2 || !(row_id&flag2)){
88                 if(!(row_id&flag)){
89                     CRTS_ssync_peer(pe_id+pep);
90                     CRTS_rma_iget(&buf_s, &rma_rplyl, sizeof(double), pe_id+pep,
91 &sum_s, &rma_rplyr);
92                     R_COUNTL++;
93                     CRTS_rma_wait_value(&rma_rplyl, R_COUNTL);
94
95                     sum_s += buf_s;
96                 }else{
97                     CRTS_ssync_peer(pe_id-pep);
98                     R_COUNTTR++;
99                     CRTS_rma_wait_value(&rma_rplyr, R_COUNTTR);
100                 }
101             }
102         }
103
104         /*8. slave 0 put sum back to host */
105         if(pe_id == 0){
106             CRTS_dma_iput(para_s.sum, &sum_s, sizeof(double), &dma_rply);
107             D_COUNT++;
108             CRTS_dma_wait_value(&dma_rply, D_COUNT);
109         }
110     }

```

编译与链接

```

sw9gcc -mftz -mieee -c -o main.o main.c
sw9gcc -mftz -mieee -c -o master.o master.c
sw9gcc -mslave -mieee -msimd -c -o slave.o slave.c
sw9gcc -mhybrid -o test main.o master.o slave.o

```

提交运行

```

bsub -I -b -q 队列名 -n 1 -shared -cgsp 64 -share_size 7000 ./test

```

4.3 LDM 共享

如 2.4 节所述，从核 LDM 空间的一部分可以配置成连续共享从而实现数据在从核阵列内多种模式的分布存储和共享访问。在连续共享过程中，支持单簇、双簇、四簇、全阵列簇、全阵列行等 5 种模式。

DMA、RMA 操作不能使用连续共享段的访问方式。

采用单簇模式时，连续共享段的 LDM 拼接方式如图 4-1 所示，图中绿色背景为一个独立的共享区间。从核簇中四个从核 LDM 的连续共享段，按图中数字顺序进行连续编址，每个从核可见本簇内的整个连续共享段。

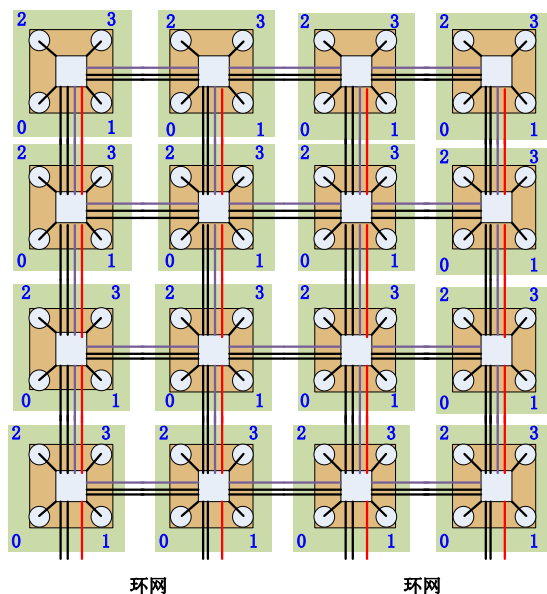


图 4-1 连续共享段（单簇模式）

采用双簇模式时，连续共享段的 LDM 拼接方式如图 4-2 所示，图中绿色背景为一个独立的共享区间。相邻两个从核簇中八个从核 LDM 的连续共享段，按图中数字顺序进行连续编址，每个从核可见两个簇内的整个连续共享段。

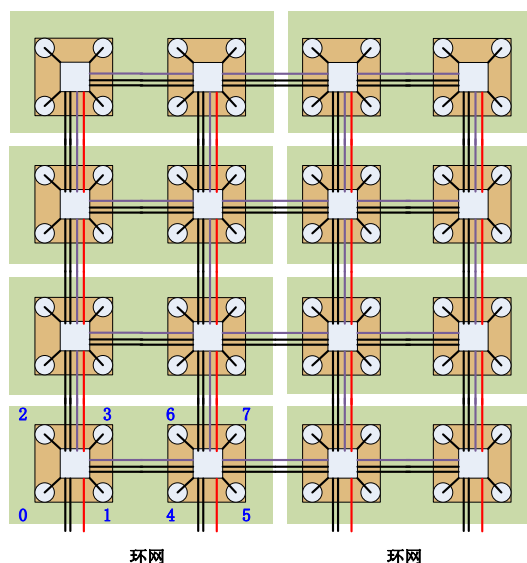


图 4-2 连续共享段（双簇模式）

采用四簇模式时，连续共享段的 LDM 拼接方式如图 4-3 所示，图中绿色背景为一个独立的共享区间。两行 16 个从核的 LDM 的连续共享段，按图中数字顺序进行连续编址，每个从核可见四个从核簇的整个连续共享段。

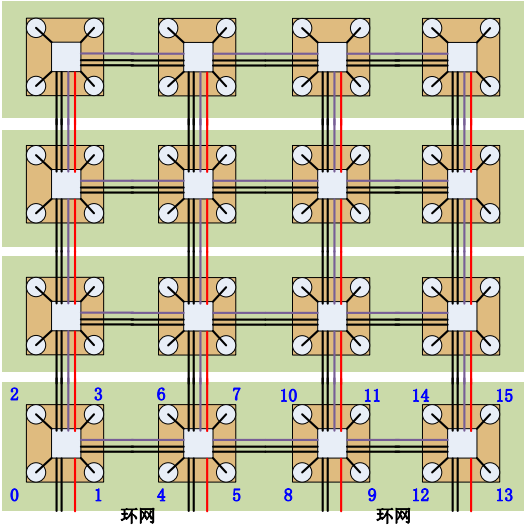


图 4-3 连续共享段（四簇模式）

采用全阵列簇模式时，连续共享段的 LDM 拼接方式如下图，图中绿色背景为一个独立的共享区间。阵列内所有从核的 LDM 的连续共享段，按图中数字顺序进行连续编址，每个从核可见阵列内的整个连续共享段。

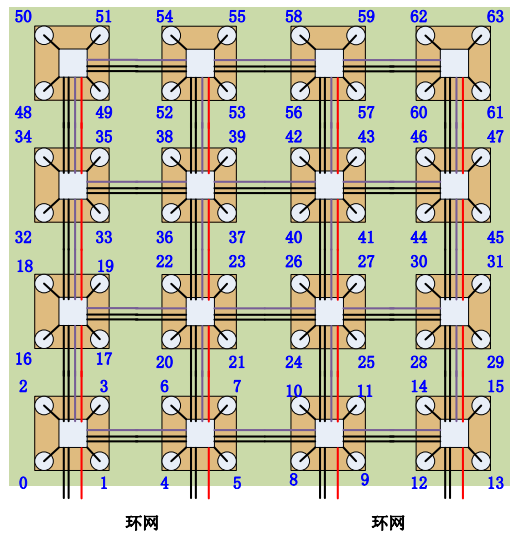


图 4-4 连续共享段（全阵列簇模式）

采用全阵列行模式时，连续共享段的 LDM 拼接方式如图 4-5 所示，图中绿色背景为一个独立的共享区间。阵列内所有从核的 LDM 的连续共享段，按图中数字顺序进行连续编址，每个从核可见阵列内的整个连续共享段。

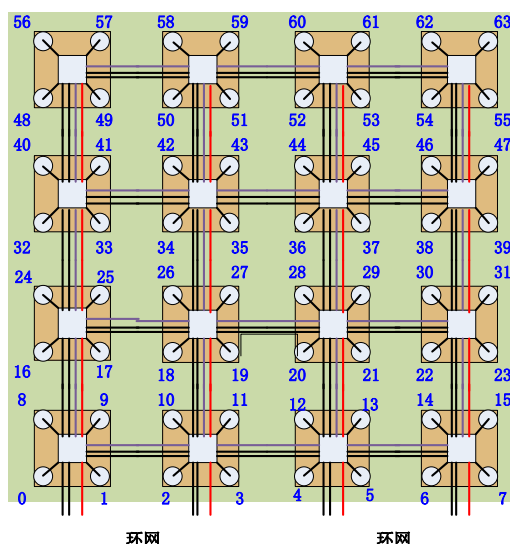


图 4-5 连续共享段（全阵列行模式）

在不同的共享模式下，每个从核只能看到其所在的共享区间（上图中绿色背景区域）的连续共享段，有效的虚地址为从核 LDM 连续共享段 0 地址，到“共享区间内从核数*每个从核连续共享段容量-1”，超出该容量限制将产生数据流异常。

表 4-1 各种连续共享模式有效访问空间大小

连续共享模式	有效访问空间大小
单簇	4*每个从核连续共享段容量
双簇	8*每个从核连续共享段容量
四簇	16*每个从核连续共享段容量
全阵列簇/全阵列行	64*每个从核连续共享段容量

4.3.1 LDM 共享模式设置方法

通过提交命令（bsub）的 `-ldm_share_mode` 选项设置 LDM 共享模式，通过 `-ldm_share_size` 选项设置每个从核内用于 LDM 共享的 LDM 空间大小（单位为 KB），每个从核内用于 LDM 共享的 LDM 空间大小只能是 0、4、8、16、32、64、128KB。

设置共享模式：

`-ldm_share_mode 0/1/2/3/4/5`

其中，

`-ldm_share_mode 0`：单簇模式

`-ldm_share_mode 1`：双簇模式

`-ldm_share_mode 2`：四簇模式

`-ldm_share_mode 4`：全阵列簇模式

`-ldm_share_mode 5`：全阵列行模式

设置每个从核内用于 LDM 共享的 LDM 空间大小（单位为 KB）：

-ldm_share_size 0/4/8/16/32/64/128

4.3.2 使用方法

1、在从核函数内声明 LDM 共享变量

在 LDM 变量前添加__thread_local_share 修饰符将变量声明为 LDM 共享变量，如下：

```
__thread_local_share double A_s[WA*HA] __attribute__((aligned(64)));  
__thread_local_share double B_s[WB*WA] __attribute__((aligned(64)));
```

2、通过 DMA 方式将数据从主存加载到共享 LDM 空间内

函数接口：

int CRTS_memcpy_sldm (void *dst, void *src, int len, int direction);

函数说明：

从核发起共享 ldm 空间的 DMA 数据传输，需要 LDM 共享空间内的所有从核调用。

参数说明：

dst: 目标地址（4B 对界/对齐）；

src: 源地址（4B 对界/对齐）；

len: 数据传输量（4B 整数倍）；

direction: 数据传输的方向，可选项有 MEM_TO_LDM 和 LDM_TO_MEM。

返回值：

成功：0；

失败：非 0。

4.3.3 程序示例

对于 3.4 节中矩阵矩阵乘示例，若矩阵 A 和矩阵 B 过大而无法装入 LDM 时，可以使用 LDM 共享方式，将 A 和 B 存入 LDM 共享空间进行计算，从核函数如下所示：

```
1  #include <crts.h>  
2  
3  #define BLOCK_SIZE 16  
4  #define WA 4*BLOCK_SIZE  
5  #define HA 4*BLOCK_SIZE  
6  #define WB 5*BLOCK_SIZE  
7
```

```

8   typedef struct {
9       int width;
10      int height;
11      int stride;
12      double* elements;
13  } Matrix;
14
15  typedef struct{
16      int widthA;
17      int heightA;
18      int widthB;
19      int heightB;
20      double* eA;
21      double* eB;
22      double* eC;
23  } Para;
24
25  #define WC WB/8
26  #define HC HA/8
27
28  __thread_local double C_s[HC][WC] __attribute__((aligned(64)));
29
30  __thread_local crts_rply_t dma_rply = 0;
31  __thread_local unsigned int D_COUNT = 0;
32
33  __thread_local_share double A_s[HC][WA] __attribute__((aligned(64)));
34  __thread_local_share double B_s[WA][WC] __attribute__((aligned(64)));
35
36  void MatrixMulSW(void* para){
37      Para para_s;
38      int rstartC, cstartC;
39      int wA, wB;
40      int i, j, e;
41
42      rstartC = CRTS_rid * HC;
43      cstartC = CRTS_cid * WC;
44
45      /* 1. get parameters from host */
46      CRTS_dma_iget(&para_s, para, sizeof(Para), &dma_rply);
47      D_COUNT++;
48      CRTS_dma_wait_value(&dma_rply, D_COUNT);
49
50      wA = para_s.widthA;
51      wB = para_s.widthB;

```

```

52
53     for(i=0; i<HC; i++)
54         for(j=0; j<WC; j++)
55             C_s[i][j] = 0.0;
56
57     /* 2. get A_s and B_s from host */
58     CRTS_memcpy_sldm(A_s, para_s.eA, WA*HA*8, MEM_TO_LDM);
59     CRTS_memcpy_sldm(B_s, para_s.eB, WB*WA*8, MEM_TO_LDM);
60
61     /* 3. coculate C */
62     for(i=0; i<HC; i++){
63         for(j=0; j<WC; j++){
64             for(e=0; e<WA; e++){
65                 C_s[i][j] += A_s[rstartC+i*WA+e]
66                             * B_s[cstartC+e*WC+j];
67             }
68         }
69     }
70
71     /* 3. put C back to host */
72     CRTS_dma_iput_stride(para_s.eC+rstartC*wB+cstartC, &C_s[0][0],
73                         HC*WC*sizeof(double), WC*sizeof(double), (wB-WC)*sizeof(double),
74                         &dma_rply);
75     D_COUNT++;
76     CRTS_dma_wait_value(&dma_rply, D_COUNT);
77 }

```

提交运行：

```

bsub -b -I -o out.log -q 队列名 -n 1 -cgsp 64 -ldm_share_mode 5 -
ldm_share_size 8 -share_size 7000 ./test

```

4.4 手工 SIMD 优化

灵活使用从核内的向量寄存器是提升从核计算性能的重要方法，神威从核提供了丰富的手工 SIMD 优化接口，完整的接口说明见《基础编译器 SIMD 用户编程手册》。以下是用手工向量化实现的从核矩阵转置的程序示例。

4.4.1 算法描述与接口

从核矩阵转置主要通过向量混洗接口实现，接口如下：

函数名：

simd_vshfw—— 字向量全混洗

参数说明：

intv8 va;

intv8 vb;

double fc;

返回值

intv8 vd

功能说明

操作数寄存器为 Va、Vb、Fc 和 Vd。Vd 的八个 32bit 的数由 Va 和 Vb 装填，具体如何装填由 Fc 说明：

Fc 的<31:28>指出 vd 的<255:224>来自 Va 或 Vb 的位置

Fc 的<27:24>指出 vd 的<223:192>来自 Va 或 Vb 的位置

Fc 的<23:20>指出 vd 的<191:160>来自 Va 或 Vb 的位置

Fc 的<19:16>指出 vd 的<159:128>来自 Va 或 Vb 的位置

Fc 的<15:12>指出 vd 的<127:96>来自 Va 或 Vb 的位置

Fc 的<11:8>指出 vd 的<95:64>来自 Va 或 Vb 的位置

Fc 的<7:4>指出 vd 的<63:32>来自 Va 或 Vb 的位置

Fc 的<3:0>指出 vd 的<31:0>来自 Va 或 Vb 的位置

Fc 相应的 4bit 位置的最高位为“0”时，Vd 相应的数来自 Va；为“1”时，Vd 相应的数来自 Vb。Va/Vb 分成自然对界的 8 个 32bit，Fc 相应的 4bit 位置的低 3bit (0~7) 用于指示是第几个 32bit 数据，为“0”表示第 0 个 32bit ([31:0])，依次类推。

通过向量混洗接口可以实现一个向量内各分量的位置交换，利用向量混洗接口实现从核矩阵转置过程如图 4-7 所示：

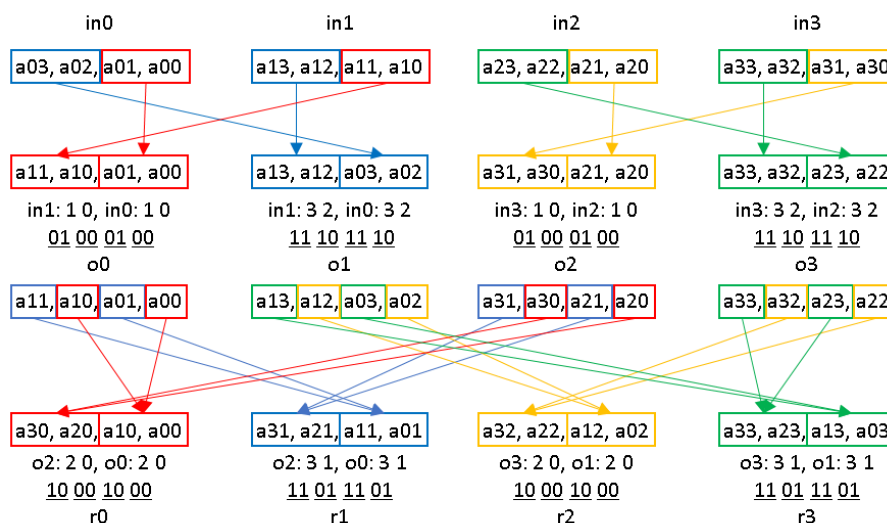


图 4-7 向量混洗接口实现矩阵转置示意图

4.4.2 程序示例

主程序

```
1  #include <crt.h>
2
3  #define BLOCK_SIZE 16
```

```

4  #define WA 4*BLOCK_SIZE
5  #define HA 4*BLOCK_SIZE
6
7  extern void transd(double *dataout, double *datain, int m, int n);
8
9  int main(int argc, char* argv[]){
10     double *in, *out;
11     int i, j, k;
12     CRTS_init();
13
14     in = (double*)malloc(WA*HA*sizeof(double));
15     out = (double*)malloc(64*HA*WA*sizeof(double));
16
17     for(i=0; i<HA; i++)
18         for(j=0; j<WA; j++)
19             in[i*WA+j] = 1.0*i + 0.01*j;
20
21     transd(out, in, HA, WA);
22
23     /* output reslut */
24     for(k=0; k<64; k++){
25         printf("slave %d:\n", k);
26         for(i=0; i<WA; i++){
27             for(j=0; j<HA; j++)
28                 printf("%f ", out[k*WA*HA+i*HA+j]);
29             printf("\n");
30         }
31     }
32
33     free(in);
34     free(out);
35     athread_halt();
36     return 0;
37 }

```

主核程序

```

1  #include <crts.h>
2
3  typedef struct{
4     double* dataout;
5     double* datain;

```

```

6    int m;
7    int n;
8    } Para;
9
10   extern SLAVE_FUN(trans_sd)(void*);
11
12   void transd(double *dataout, double *datain, int m, int n){
13       Para para;
14       para.dataout = dataout;
15       para.datain = datain;
16       para.m = m;
17       para.n = n;
18
19       athread_spawn(trans_sd, &para);
20       athread_join();
21   }

```

从核程序

```

1    #include <crt.h>
2    #include <simd.h>
3
4    typedef struct{
5        double* dataout;
6        double* datain;
7        int m;
8        int n;
9    } Para;
10
11    __thread_local double datain_s[1024] __attribute__((aligned(64)));
12    __thread_local double dataout_s[1024] __attribute__((aligned(64)));
13
14    __thread_local unsigned long arg1[8] __attribute__((aligned (512))) =
    {0x85A9289CA3018820,0xDEB3,0xC7B9ACBDAB4398A4,0xFFBB,0x8
    6B128AD23029020,0xEF33};
15    __thread_local unsigned long arg2[8] __attribute__((aligned (512))) =
    {0xA7B96ABDA7239862,0xFFB7,0x49CA30398A418820,0xBDAB,0xC
    DEB387B9AC5A928,0xFFBB};
16    __thread_local intv16 v0,v1,v2,v3,v4,v5,v6,v7,o0,o1,o2,o3,o4,o5,o6,o7;
17    __thread_local intv16 a1,a2;
18    __thread_local int rb = 128;
19

```

```

20  __thread_local crts_rply_t dma_rply = 0;
21  __thread_local unsigned int D_COUNT = 0;
22
23  static inline void trans88d(double* dataout, double* datain, int i, int j, int m,
int n){
24      simd_loadu(v0, (int*)(datain +      i*n+j));
25      simd_loadu(v1, (int*)(datain + (i+1)*n+j));
26      simd_loadu(v2, (int*)(datain + (i+2)*n+j));
27      simd_loadu(v3, (int*)(datain + (i+3)*n+j));
28      simd_loadu(v4, (int*)(datain + (i+4)*n+j));
29      simd_loadu(v5, (int*)(datain + (i+5)*n+j));
30      simd_loadu(v6, (int*)(datain + (i+6)*n+j));
31      simd_loadu(v7, (int*)(datain + (i+7)*n+j));
32
33      simd_load(a1, arg1);
34      o0 = simd_vshfw(v0, v1, a1);
35      o2 = simd_vshfw(v2, v3, a1);
36      o4 = simd_vshfw(v4, v5, a1);
37      o6 = simd_vshfw(v6, v7, a1);
38
39      *((uint512*)&a2) = simd_srlx(*((uint512*)&a1), rb);
40      o1 = simd_vshfw(v0, v1, a2);
41      o3 = simd_vshfw(v2, v3, a2);
42      o5 = simd_vshfw(v4, v5, a2);
43      o7 = simd_vshfw(v6, v7, a2);
44
45      *((uint512*)&a1) = simd_srlx(*((uint512*)&a2), rb);
46      v0 = simd_vshfw(o0, o2, a1);
47      v2 = simd_vshfw(o1, o3, a1);
48      v4 = simd_vshfw(o4, o6, a1);
49      v6 = simd_vshfw(o5, o7, a1);
50
51      simd_load(a2, arg2);
52      v1 = simd_vshfw(o0, o2, a2);
53      v3 = simd_vshfw(o1, o3, a2);
54      v5 = simd_vshfw(o4, o6, a2);
55      v7 = simd_vshfw(o5, o7, a2);
56
57      *((uint512*)&a1) = simd_srlx(*((uint512*)&a2), rb);
58      o0 = simd_vshfw(v0, v4, a1);
59      o1 = simd_vshfw(v1, v5, a1);
60      o2 = simd_vshfw(v2, v6, a1);
61      o3 = simd_vshfw(v3, v7, a1);
62

```

```

63      *((uint512*)&a2) = simd_srlx(*((uint512*)&a1), rb);
64      o4 = simd_vshfw(v0, v4, a2);
65      o5 = simd_vshfw(v1, v5, a2);
66      o6 = simd_vshfw(v2, v6, a2);
67      o7 = simd_vshfw(v3, v7, a2);
68
69      simd_storeu(o0, (int*)(dataout +      j*m+i));
70      simd_storeu(o1, (int*)(dataout + (j+1)*m+i));
71      simd_storeu(o2, (int*)(dataout + (j+2)*m+i));
72      simd_storeu(o3, (int*)(dataout + (j+3)*m+i));
73      simd_storeu(o4, (int*)(dataout + (j+4)*m+i));
74      simd_storeu(o5, (int*)(dataout + (j+5)*m+i));
75      simd_storeu(o6, (int*)(dataout + (j+6)*m+i));
76      simd_storeu(o7, (int*)(dataout + (j+7)*m+i));
77  }
78
79  void trans_sd(Para *para){
80      Para para_s;
81      int i, j, m, n;
82
83      /* 1. get parameters from host */
84      CRTS_dma_iget(&para_s, para, sizeof(Para), &dma_rply);
85      D_COUNT++;
86      CRTS_dma_wait_value(&dma_rply, D_COUNT);
87
88      m = para_s.m;
89      n = para_s.n;
90
91      /* 2. get datain from host */
92      CRTS_dma_iget(datain_s,      para_s.datain,      m*n*sizeof(double),
93      &dma_rply);
94      D_COUNT += 1;
95      CRTS_dma_wait_value(&dma_rply, D_COUNT);
96
97      /* 3. trans datain */
98      for(i=0; i<(((m+7)>>3)-1); i++){
99          for(j=0; j<(((n+7)>>3)-1); j++){
100              trans88d(dataout_s, datain_s, i<<3, j<<3, m, n);
101              trans88d(dataout_s, datain_s, i<<3, n-8, m, n);
102          }
103      }
104      if((i<<3)+8 == m){
105          for(j=0; j<(((n+7)>>3)-1); j++){
106              trans88d(dataout_s, datain_s, i<<3, j<<3, m, n);

```

```

106     trans88d(dataout_s, datain_s, i<<3, n-8, m, n);
107 }else{
108     for(i=i<<3; i<m; i++){
109         for(j=0; j<n; j++)
110             dataout_s[j*m+i] = datain_s[i*n+j];
111     }
112 }
113
114 /* 3. put dataout back to host */
115     CRTS_dma_iput(para_s.dataout+CRTS_tid*n*m,          dataout_s,
n*m*sizeof(double), &dma_rply);
116     D_COUNT++;
117     CRTS_dma_wait_value(&dma_rply, D_COUNT);
118 }

```

编译与链接

```

sw9gcc -mftz -mieee -c -o main.o main.c
sw9gcc -mftz -mieee -c -o master.o master.c
sw9gcc -mslave -mieee -msimd -c -o slave.o slave.c
sw9gcc -mhybrid -static -o test main.o master.o slave.o

```

提交运行

```

bsub -b -I -o out.log -q 队列名 -shared -n 1 -cgsp 64 -share_size 7000 ./test

```

4.5 自动 SIMD 优化

从核支持 512 位向量扩展, 充分利用 SIMD 优化可以有效提升神威众核程序性能。考虑 3.5 节中的分块矩阵矩阵乘程序, 在计算每一个 C 分块时, 对 B 的列块的访问是不连续的, 若在完成 B 的列块加载之后, 加载的 B 列块进行转置, 则可以将对 B 的列块的跨步访存转换为连续访存, 从而实现计算的自动向量化优化, 如下所示:

```

1  .....
2  /* block A row block and B column block */
3  for(ibA=0; ibA<wA; ibA+=BAMAX){
4      bA = BAMAX;
5      if(ibA+bA > wA) bA = wA - ibA;
6
7      /* 2. get A_s and B_s from host */
8      CRTS_dma_iget_stride(A_s, para_s.eA+rstartC*wA+ibA,
9      iend*bA*sizeof(double), bA*sizeof(double), (wA-bA)*sizeof(double),

```

```

10  &dma_rply);
11  D_COUNT++;
12  CRTS_dma_iget_stride(B_s, para_s.eB+cstartC+ibA*wB,
13  bA*jend*sizeof(double), jend*sizeof(double), (wB-jend)*sizeof(double),
14  &dma_rplyB);
15  D_COUNTB++;
16  CRTS_dma_wait_value(&dma_rplyB, D_COUNTB);
17
18  trans_sd(Bt_s, B_s, bA, wblock);
19
20  CRTS_dma_wait_value(&dma_rply, D_COUNT);
21
22
23  /* 3. coculate C */
24  for(i=0; i<iend; i++){
25      for(j=0; j<jend; j++){
26          for(e=0; e<bA; e++){
27              C_s[i*jend+j] += A_s[i*bA+e]
28                          * Bt_s[i*bA+e];
29          }
1      }
2  }
3  }
4  .....

```

程序第 18 行对 B_s 进行转置存于 Bt_s，使得第 27、28 行的计算中对 Bt_s 的访问为连续访问，第 27、28 行可以进行自动向量化优化。

4.6 离散访存库

4.6.1 函数接口

void DIS_prePosmod(int *iter_table,int *pos_table, int iter_size, int data_span, struct DIS_TYPEDATA *pos2global);

函数说明：

对离散数据下标进行预处理，建立坐标映射模型。对于计算过程中离散下标保持不变的数组，在进行迭代计算之前进行一次下标预处理，在计算过程中可反复使用预处理模型。

参数说明：

iter_table: 存储离散数据下标所对应的循环体下标；

pos_table: 存储离散数据的下标；

iter_size: 离散数据总量；

data_span: 离散数据下标的宽度；

pos2global: 预处理后生成的坐标映射模型（输出）；

`void DIS_preprocess_join();`

函数说明：

主核上的进程以阻塞方式调用该函数，直到从核预处理执行完毕。

`void DIS_read(void *original_master_data, int slave_block_i, int vec_size, void struct DIS_TYPEDATA *pos2global, void *result_slave_data, )`

函数说明：

将本地从核计算所需的原始主存离散数据块保存至本地 LDM 缓存空间中（对应：result_slave_data 变量），使原始离散数据变为本地连续数据块；

参数说明：

original_master_data: 主存原始离散数据；

slave_block_i: 对应数据分块编号；

vec_size: 对应向量块长度；

pos2global: 坐标映射模型；

result_slave_data: 存储本地计算所需数据；

`void DIS_write(void *updata_table, int slave_block_i, int vec_size, struct DIS_TYPEDATA *pos2global, void *updata_master_data);`

函数说明：

将本地 LDM 缓存内计算后所需更新的离散数据批量更新至主存离散数据表中；

参数说明：

updata_table: 需要更新的本地离散数据块；

slave_block_i: 对应数据分块编号；

vec_size: 对应向量块长度；

pos2global: 坐标映射模型；

updata_master_data: 需要更新的主存目标数据；

4.6.2 程序示例

一个一维离散访存的示例如下：

合并的主函数与主核函数

```
1  #include <crts.h>
2  #include <yyj_lib.h>
3
4  #define N 40960
5  #define M 360000
6
7  typedef struct{
8      double *A;
9      double *y;
```

```

10     struct DIS_TYPEDATA *m_buff;
11 } Para;
12
13 uint64_t my_rand(){
14     uint64_t x=1,y=7;
15     x ^= x<<29;
16     x ^= x>>23;
17     x ^= y ^ (y >> 31);
18     return x+y;
19 }
20
21 extern SLAVE_FUN(func)(Para *para);
22
23 int main(int argc, char* argv[]){
24     double A[N];
25     int idx[M];
26     double *ym, *ys;
27     int *row;
28     struct DIS_TYPEDATA m_buff;
29     int i, vt, tag;
30
31     Para para;
32
33     DIS_init();
34
35     for(i=0; i<M; i++){
36         idx[i] = my_rand() % N;
37     }
38     for(i=0; i<N; i++){
39         vt = my_rand() % N;
40         A[i] = 3.14*i + 0.27 * vt;
41     }
42
43     ym = (double*)malloc(M*sizeof(double));
44     ys = (double*)malloc(M*sizeof(double));
45     memset(ym, 0, M*sizeof(double));
46     memset(ys, 0, M*sizeof(double));
47
48     /* coculate in master */
49     for(i = 0; i < M; i++){
50         ym[i] += A[idx[i]];
51     }
52
53     row = (int*)malloc((M+1)*sizeof(int));

```

```

54     for(i = 0; i <= M; i++)
55         row[i] = i;
56
57     DIS_prePosmod(&row[0], &idx[0], M, N, &m_buff);
58     DIS_pre_join();
59
60     /* coculate in slave */
61     para.A = A;
62     para.y = ys;
63     para.m_buff = &m_buff;
64     athread_spawn(func, &para);
65     athread_join();
66
67     tag=0;
68     for(i=0; i<M; i++){
69         if(ym[i] != ys[i]){
70             tag=1;
71             break;
72         }
73     }
74
75     if(tag == 1)
76         printf("result err! err idx :%d, err result :%lf, right result: %lf\n", i, ys[i],
77             ym[i]);
78     else
79         printf("slave cal right\n");
80     free(ym);
81     free(ys);
82     athread_halt();
83     return 0;
84 }

```

从核函数

```

1     #include <string.h>
2     #include <crts.h>
3     #include <yyj_pre_def.h>
4     #include <yyj_rw.h>
5
6     #define N 40960
7     #define M 95000

```

```

8
9     typedef struct{
10         double *A;
11         double *y;
12         struct DIS_TYPEDATA *m_buff;
13     } Para;
14
15     __thread_local crts_rply_t dma_rply = 0;
16     __thread_local unsigned int D_COUNT = 0;
17
18     void func(Para *para){
19         Para para_s;
20         struct DIS_TYPEDATA m_buff_s;
21         int loop_size, block_st, val_table_size, i_block, pos_size, local_size,
loop_start;
22         double tmp;
23         double *sum;
24
25         int block_n, block_res;
26         int end, idx_len, idx_byte, j_st, j_ed, size_end, ii, jj, jj_st, jj_ed, j9, i_pos;
27         int *idx;
28         double *vp_s;
29
30         CRTS_dma_iget(&para_s, para, sizeof(Para), &dma_rply);
31         D_COUNT++;
32         CRTS_dma_wait_value(&dma_rply, D_COUNT);
33         CRTS_dma_iget(&m_buff_s, para_s.m_buff, sizeof(struct
DIS_TYPEDATA), &dma_rply);
34         D_COUNT++;
35         CRTS_dma_wait_value(&dma_rply, D_COUNT);
36
37         loop_size = m_buff_s.loop_size;
38         val_table_size = m_buff_s.pos_span;
39         sum = (double*)ldm_malloc(_dis_dblock_*sizeof(double));
40
41         DIS_Area(loop_size, &local_size, &loop_start);
42         block_n = (local_size + _dis_dblock_ - 1)/_dis_dblock_;
43         block_res = local_size % _dis_dblock_;
44         if(block_res == 0) block_res = _dis_dblock_;
45
46         for(i_block=0; i_block<block_n; i_block++){
47             block_st = loop_start + i_block*_dis_dblock_;
48             end = (i_block == block_n-1)? block_st + block_res : block_st +
_dis_dblock_;

```

```

49     idx_len = end - block_st + 1 ;
50     idx_byte = idx_len*sizeof(int);
51     idx = (int *)ldm_malloc(idx_byte);
52
53     CRTS_dma_iget(&idx[0], &(m_buff_s.loop_table[block_st]), idx_byte,
54 &dma_rply);
55     D_COUNT++;
56     CRTS_dma_wait_value(&dma_rply, D_COUNT);
57
58     j_st = idx[0];
59     size_end = idx[idx_len-1];
60     pos_size = size_end - j_st;
61
62     vp_s = (double *)ldm_malloc(pos_size*sizeof(double));
63     memset(&vp_s[0], 0, pos_size*sizeof(double));
64
65     yyj_readD(para_s.A, &vp_s[0], i_block, pos_size, 1, &m_buff_s);
66
67     i_pos=0;
68     jj_st = idx[0];
69     for(ii=block_st; ii<end; ii++){
70         jj_ed = idx[ii+1-block_st];
71         tmp=0.;
72         for(jj=jj_st; jj<jj_ed; jj++){
73             tmp += vp_s[i_pos];
74             i_pos++;
75         }
76         sum[ii-block_st] = tmp;
77         jj_st = jj_ed;
78     }
79
80     CRTS_dma_iput(&(para_s.y[block_st]),      &sum[0],      (idx_len-
1)*sizeof(double), &dma_rply);
81     D_COUNT++;
82     ldm_free(&idx[0], idx_byte);
83     ldm_free(&vp_s[0], pos_size*sizeof(double));
84     CRTS_dma_wait_value(&dma_rply, D_COUNT);
85 }
86
87 ldm_free(&sum[0], _dis_dblock_*sizeof(double));
88 }

```